☑ 1. Data Preprocessing

Data preprocessing is essential to clean and prepare data before feeding it into a machine learning model.

- **Steps:**
  - Handling missing values
  - Encoding categorical variables
  - Feature scaling
- **Example:**

python
CopyEdit

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.impute import SimpleImputer

# Sample dataset
data = pd.DataFrame({
    'Age': [25, 27, 29, None, 22],
    'Gender': ['Male', 'Female', 'Female', 'Male', 'Female'],
    'Salary': [50000, 54000, 58000, 60000, None]
})

# Handle missing values
imputer = SimpleImputer(strategy='mean')
data[['Age', 'Salary']] = imputer.fit_transform(data[['Age', 'Salary']])

# Encode categorical variable
le = LabelEncoder()
data['Gender'] = le.fit_transform(data['Gender'])  # Male=1, Female=0

# Feature Scaling
scaler = StandardScaler()
data[['Age', 'Salary']] = scaler.fit_transform(data[['Age', 'Salary']])

print(data)
```

☑ 2. Model Selection and Evaluation

Choose the best model using **training/test split**, **cross-validation**, and evaluate using metrics like **accuracy**, **precision**, **recall**, **F1-score**.

🔹 Example (Classification):

python
CopyEdit

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.ensemble import RandomForestClassifier

# Features and labels
X = data[['Age', 'Salary', 'Gender']]
y = [1, 0, 0, 1, 0]  # Sample target (binary classification)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Model
model = RandomForestClassifier()
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

☑ 3. Supervised Learning Algorithms

📌 Regression Example (Linear Regression):

python
CopyEdit

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Dataset
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 4, 6, 8, 10])

model = LinearRegression()
model.fit(X, y)
```

```
# Predict
print(model.predict([[6]]))  # Output: 12
```

📌 Classification Example (Logistic Regression):

```
python
CopyEdit
from sklearn.linear_model import LogisticRegression

# Sample binary classification
X = [[0], [1], [2], [3]]
y = [0, 0, 1, 1]

model = LogisticRegression()
model.fit(X, y)

print(model.predict([[1.5]]))  # Output: [0] or [1]
```

☑ 4. Unsupervised Learning Algorithms

📌 Clustering Example (KMeans):

```
python
CopyEdit
from sklearn.cluster import KMeans

data = [[1, 2], [1, 4], [1, 0],
        [10, 2], [10, 4], [10, 0]]

kmeans = KMeans(n_clusters=2)
kmeans.fit(data)

print("Cluster Centers:", kmeans.cluster_centers_)
print("Labels:", kmeans.labels_)
```

📌 Dimensionality Reduction (PCA):

```
python
CopyEdit
from sklearn.decomposition import PCA
import numpy as np
```

```
# Sample 3D data
X = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X)

print("Reduced Data:\n", X_reduced)
```

☑ 5. Pipelines

Pipelines help streamline the preprocessing and modeling steps.

- **Example:**

```
python
CopyEdit
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# Sample dataset
X = [[0.5], [2.5], [1.0], [3.5]]
y = [0, 1, 0, 1]

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', LogisticRegression())
])

pipeline.fit(X, y)
print(pipeline.predict([[1.5]]))
```